

変数と代入

メモリモデルで理解する

rkskmt

1

事件：関数に渡したただけなのに

成績データの「2倍にしたコピー」を作ろうとした。

```
Code
1 def double_all(data):
2     for i in range(len(data)):
3         data[i] = data[i] * 2
4     return data
5
6 scores = [10, 20, 30]
7 result = double_all(scores)
8 print(result) # [20, 40, 60] ← 期待どおり
9 print(scores) # [20, 40, 60] ← !? 元データまで壊れた
```

- **scores** には一度も代入していないのに、**元の成績が消えた**

🚨 今回のゴール

この回が終わる頃には、この事故を **図1枚で説明** でき、**1行で防げる** ようになる。

2

リテラル（値）

- 事故の正体を突き止めるために、敢えて「変数と代入」をメモリモデルからおさらいする
- Pythonでは**値**を直接コードに書ける
 - 値はそれだけで評価できる（エラーにならない）
 - 変数や関数なしで、その行に書くだけで意味を持つ

```
Code
1 1
2 "1"
3 [1, 2, 3]
4 True
5 1.0
```

📖 メモリ上での記述

上記の値はその行が実行された時に、**メモリ上に値として記述**される（**000001001** のようにメモリ上のスペースを使って記録される）。これらを **リテラル** と呼ぶ — 直接コード中に記述できる値の表現方法。

3

変数とは

一方、このように書くとエラーになる。なぜ？

Code

```
1 a # NameError: name 'a' is not defined
```

- **1** や **"1"** は **リテラル**（直接記述できる値）
- **a** は値ではなく **変数** として認識される



- 変数とは、**値をつかんで離さないための「名前」** — 看板が立っている限り値は消えない
- **a** はまだどこにも立てていない看板なので、エラーになる

変数の命名規則

変数として認識される単語のルールは以下のとおり。

例外はない — ここは覚えるしかない

- 文字（a-z, A-Z）またはアンダースコア（`_`）で**始まる**
 - つまり数字始まりはダメ（NG例：3200scores）
 - 終わるのはOK（例：scores_of_3200）
- 2文字目以降が文字、数字（0-9）、アンダースコアであること
- 予約語（**if / for / while** など）ではないこと

Code

```
1 a01      # OK。変数
2 _aa_aa   # OK。変数
3 1a       # NG。最初が数字
4 continue # NG。予約語
```

代入

- **変数 = リテラル** と記述すると「**代入**」という操作がおこなわれる
 - 左辺は変数
 - 右辺はリテラルかオブジェクト

Code

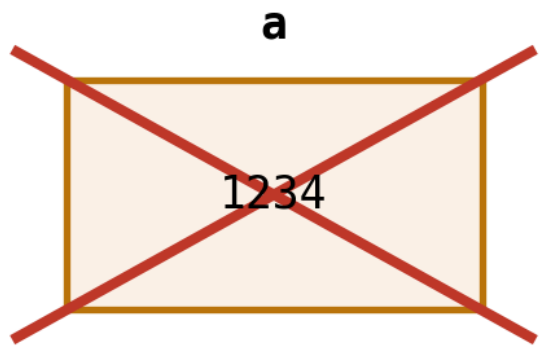
```
1 a = 1234
2 test_2 = "test2"
```

💡 箱ではなく「看板」

- **a = 1234** は「**a** という箱に **1234** が入る」 **ではない**
- **a** は **1234** が格納された住所（アドレス）を指す看板になる
- ただし、アクセス時は **その看板自体ではなく住所が指す先にあるリテラルやオブジェクトが返る**

× 箱モデル：a という箱に値が入る

○ 看板モデル：a は置き場所を指す



a



1234

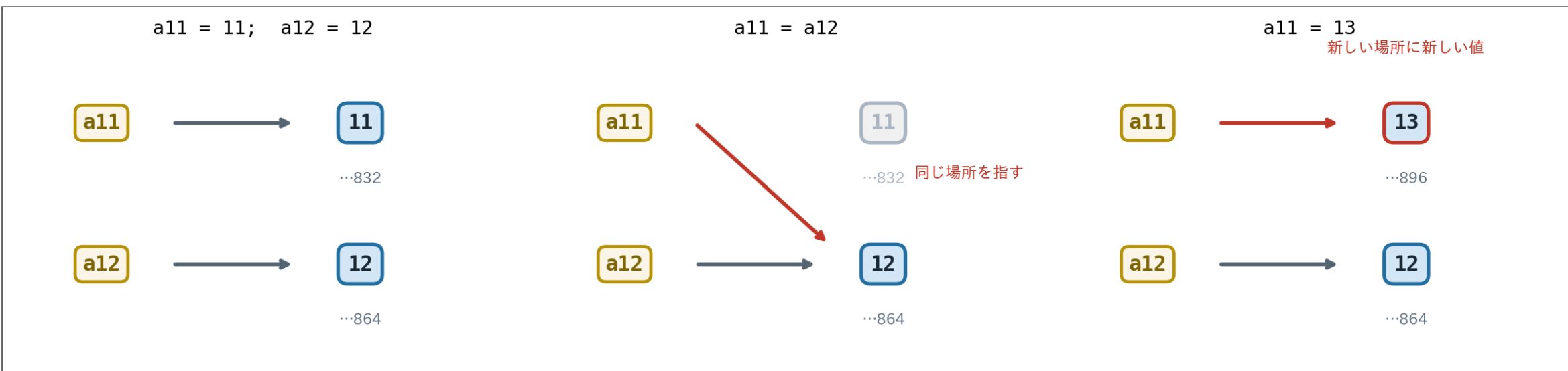
アドレス ...832

変数 = 変数 の代入

- **変数同士の代入** は、左辺に右辺のアドレスを代入する
→ つまり、**同じアドレス**を指すようになる

```
Code
1 a11 = 11
2 a12 = 12
3 a11 = a12 # a11とa12は同じアドレスを指す
4 print(a12) # 12
5
6 a11 = 13 # 別の値に変更
7 print(a11) # 当然13
8 print(a12) # ??? (同じアドレスを指しているはずだが?)
```

- 結果： **12**
- **a11 = 13** は「11 だった場所に 13 を書き込む」 **ではない** — **看板の掛け替え**
- 代入 **x = ...** は **常に掛け替え**。共有していたのは3行目から5行目までの一瞬だけ
- 5行目以降、共有していた場所を指すのは a12 だけになる



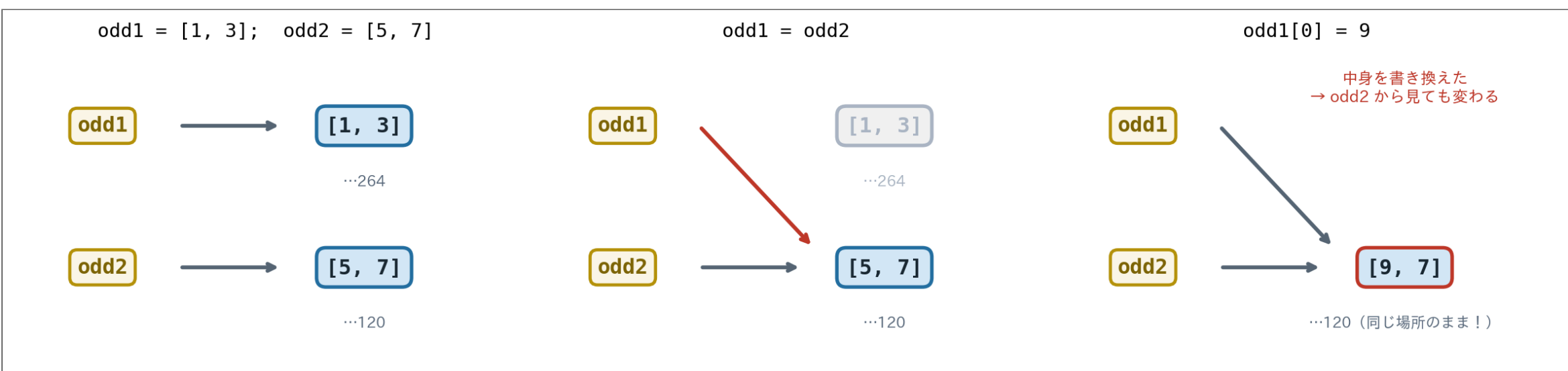
7

リストの代入（罫）

数値でやった操作をリストでやると...

```
Code
1 odd1 = [1, 3]
2 odd2 = [5, 7]
3 odd1 = odd2
4 odd1[0] = 9
5 print(odd2) # ???
```

- 結果： **[9, 7]**
- **odd1[0] = 9** は看板の掛け替え **ではない** — 指している先の **中身の変更**
- 看板は2枚とも同じリストを指したまま → **odd2** から見ても変わる



8

クイズ：どっちなら odd2 は無事？

ほぼ同じに見える2つのコード。 `print(odd2)` は何と出る？

Code

```
1 # A
2 odd1 = [1, 3]
3 odd2 = odd1
4 odd1 = [9, 7]
5 print(odd2) # ???
```

Code

```
1 # B
2 odd1 = [1, 3]
3 odd2 = odd1
4 odd1[0] = 9
5 print(odd2) # ???
```

- A： `[1, 3]` — `odd1 = [9, 7]` は **看板の掛け替え**。odd2 は無傷
- B： `[9, 7]` — `odd1[0] = 9` は **中身の変更**。odd2 も巻き込まれる



ルールは2本だけ

	操作	何が起きる
ルール1	<code>x = ...</code> （代入）	看板の掛け替え 。型に関係なく常に。コピーは起きない
ルール2	<code>x[0] = ...</code> や <code>.append()</code>	指している先の 中身を変更 。できるのは ミュータブル だけ

- 「リストだけ変な動きをする」のではない — リストは **起こせる操作（ルール2）が1つ多い** だけ
- 迷ったら「これは掛け替え？ 中身の変更？」と聞けば、すべてのクイズが解ける

💡 ヒント

この2本で、冒頭の事件も説明できる。あと少し道具（`id()`）を足してから戻ろう。

id() で住所を確認（数値）

id() 関数で、変数が指しているアドレスを調べられる。

Code

```
1 a11 = 11
2 a12 = 12
3 print(id(a11)) # 2083050422832
4 print(id(a12)) # 2083050422864
5 a11 = a12
6 print(id(a11)) # 2083050422864。a12と同じ場所
7 a11 = 13
8 print(id(a11)) # 2083050422896。違う場所を指している。
```

- 代入のたびに **指す先（アドレス）が変わっている** — ルール1（掛け替え）を **id()** で目撃できる
- **a11 = 13** をしても、**11** や **12** の場所は書き換わっていない

⚠ 豆知識：小さい整数は使い回される

- CPython は **-5~256 の int をあらかじめ作って使い回す**
- だから **x = 11; y = 11** とすると **id(x) == id(y)** は **True** になる
- 「同じ値なら同じ場所を指すことがある」だけで、看板モデル自体は変わらない（高速化のための最適化）

11

id() で住所を確認（リスト）

リストでは、要素に代入してもアドレスが変わらない。

Code

```
1 odd1 = [1, 3]
2 odd2 = [5, 7]
3 print(id(odd1)) # 2738928019264
4 print(id(odd2)) # 2738927989120
5 odd1 = odd2
6 print(id(odd1)) # 2738927989120
7 print(id(odd2)) # 2738927989120
8 odd1[0] = 9      # odd2ではなくodd1を変更
9 print(odd1)      # [9, 7]
```

- **odd1 = odd2** 後、両者が **同じアドレスを指している**
- だから **odd1[0] = 9** が **odd2** にも **影響する**

12

「同じ場所？」を直接聞く：is 演算子

- `id() == id()` を毎回書くのは回りくどい — `is` で直接聞ける

Code

```
1 odd1 = [9, 7]
2 odd2 = odd1
3
4 print(odd1 is odd2)      # True   (同じ場所を指している)
5 print(odd1 == [9, 7])    # True   (中身が同じ)
6 print(odd1 is [9, 7])    # False  (中身は同じでも別の場所)
```

- `==` は中身の比較、`is` は場所の比較
- 「この2枚の看板、同じ実体を指してる？」と聞きたいときは `is`

ミュータブル / イミュータブル

値（オブジェクト）は2種類に分かれる

- **イミュータブル (immutable)**：中身を変更する手段が**存在しない**型
 - `int` `str` `bool` `tuple` 等
 - 「変えた」ように見える操作も、すべて**新しい値への掛け替え**（ルール1）
 - だからアドレスを意識しなくても事故が起きない
- **ミュータブル (mutable)**：中身を**その場で変更できる**型（ルール2が使える）
 - `list`、`dict` 等
 - 看板を何枚立てても実体は1つ — **共有に気づかないと事故になる**

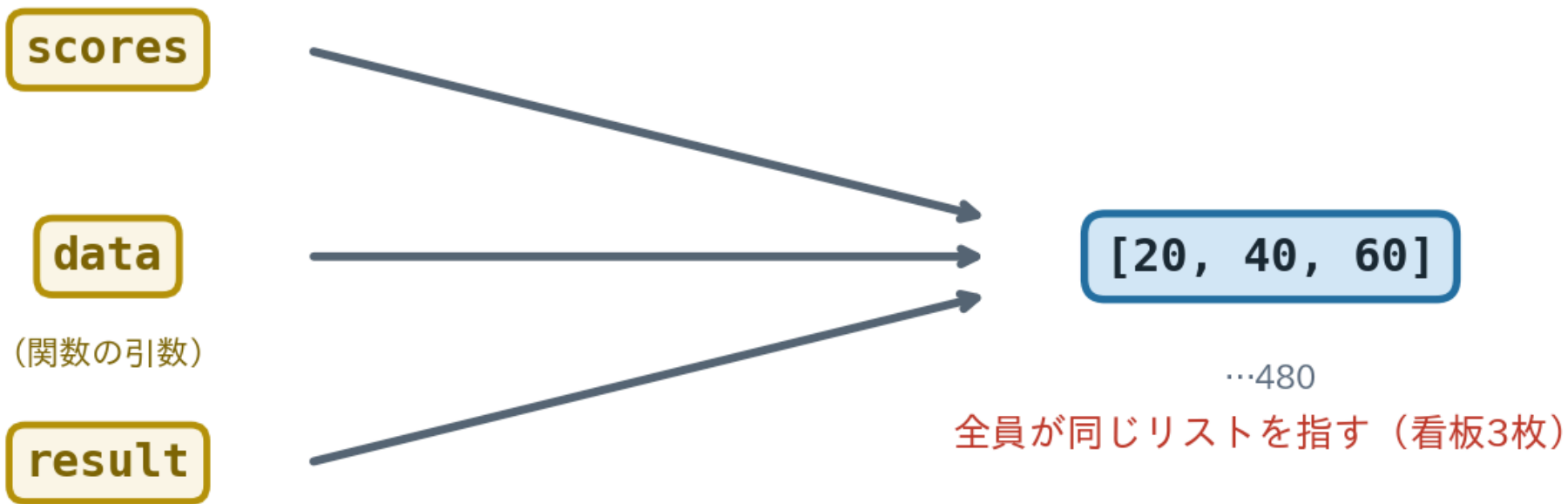
分類	型
イミュータブル	<code>int</code> / <code>float</code> / <code>complex</code> / <code>bool</code> / <code>str</code> / <code>tuple</code> / <code>frozenset</code> / <code>bytes</code> / <code>None</code>
ミュータブル	<code>list</code> <code>dict</code> <code>set</code> クラス など

事件の解決：冒頭のコードに戻る

Code

```
1 def double_all(data):
2     for i in range(len(data)):
3         data[i] = data[i] * 2    # ← ルール2：中身の変更！
4     return data
5
6 scores = [10, 20, 30]
7 result = double_all(scores)    # ← 関数に渡す= data という看板を立てる（ルール1）
8 print(result)    # [20, 40, 60]
9 print(scores)    # [20, 40, 60]
```

- 関数に渡しても **コピーは起きない**。引数 **data** は **scores** と同じリストを指す看板（ルール1）
- 中で **data[i] = ...** と **中身を変更** したので（ルール2）、**scores** から見ても壊れていた
- 約束どおり、**図1枚**で説明できる：



ミュータブルの罠を回避する：コピー

- リストを **別のアドレスに新たに複製** したいときは **.copy()** を使う
 - 「この場面を残したい」ときにセーブデータを別スロットへコピーするイメージ
 - 単に **=** で代入しても複製されない（看板が増えるだけ）
- 冒頭の事故も、**渡す側の1行**で防げる：

Code

```
1 result = double_all(scores.copy())    # 複製を渡す → scores は無傷
```

Code

```
1 odd1 = [1, 3]
2
3 # NG: 同じアドレスを共有 → 両方が変わる
4 odd2 = odd1
5 odd2[0] = 9
6 print(odd1)  # [9, 3] ← 意図せず変わってしまう
7
8 # OK: .copy() で別アドレスの複製を作る
9 odd3 = odd1.copy()
10 odd3[0] = 99
11 print(odd1)  # [9, 3] ← 変わらない
12 print(odd3)  # [99, 3]
13
14 # （参考）全スライス [:] でも同じ結果
15 odd4 = odd1[:]
16 print(id(odd1) == id(odd4))  # False (別アドレス)
```





「箱ではなく看板」の変数モデルを、次の4つの実験で確かめよ。

1. **紙に看板図を描く**：次のコードを1行ずつ、看板と矢印で描きながら追い、最後の出力を**予想してから**実行して答え合わせ

```
Code
1 a = [1, 2]
2 b = a
3 a.append(3)    # 掛け替え？ 中身の変更？
4 a = [9]        # 掛け替え？ 中身の変更？
5 print(b)       # ???
```

2. **dict** がミュータブルであることを **id()**（または **is**）で確認（**dict** の書き方は次回詳しく — ここでは次の2行だけ使う）

```
Code
1 d = {"名前": "田中"}    # dict を作る
2 d["点数"] = 80          # キーを指定して中身を変更する
```

3. **list** と同じようにアクセスできる **str** は、ミュータブルかイミュータブルかを確認
4. 上記の例で **odd1[0]** のアドレスがどうなるかを確認（**list** はミュータブルだが、**list** の要素は **int** なのでイミュータブルのはず）

▶ 解答例を見る

💡 矢印が動くのを見たいとき

Python Tutor にコードを貼ると、看板（名前）と矢印が1行ごとに動く様子をステップ実行で確認できます。今日の図と同じ世界観です。

まとめ

- 変数は値そのものではなく、**値が置かれたアドレスを指す看板**
- **ルール1**：代入 **x = ...** は型に関係なく **常に看板の掛け替え**。コピーは起きない
- **ルール2**：**ミュータブル**（list / dict 等）だけが**中身をその場で変更**できる — 共有中の変更は全員に見える
- イミュータブル（int / str / bool / tuple 等）は中身を変える手段がないので事故が起きない
- 事故を防ぐには **.copy()** で別の実体を渡す。確認の道具は **id()**（住所）と **is**（同じ場所か）